ЛАБОРАТОРНА РОБОТА № 4

МОДУЛЬНЕ ПРОГРАМУВАННЯ, ПОРОЗРЯДНІ ОПЕРАЦІЇ, БІТОВІ МАСКИ ТА PULL REQUEST У GIT

Мета: опанувати поділ програми на функції з прототипами, роботу static-змінних, що зберігають стан між викликами, порозрядні операції &, |, ^, ~, <<, >> і чотири операції над бітами регістра, а також злиття коду через Pull Request на GitHub.

Хід роботи:

Посилання на GitHub: https://github.com/VladyslavChepel/programming-sem1

Все виконано в окремій гілці lab04 , потім гілку була відправлена на GitHub, відкрий Pull Request з lab04 в main і після перевірки він був злитий.

Завдання 1. Обчислити опір двох паралельно з’єднаних резисторів, у static-змінній зберігати мінімальний опір за сесію.

Опір рахує функція paralel по формулі R1·R2/(R1 + R2), R1 і R2 передаються за значенням. Мінімум зберігається в static змінній min всередині функції min_opir. Спочатку там -1, так перший опір одразу записується як мінімум. Прототипи обох функцій стоять перед main.

Перевіряв на чотирьох парах (рис. 2). 100 і 100 Ом дали 50 Ом, 47 і 220 Ом дали 38,73 Ом, і мінімум став 38,73. Пара 1000 і 1000 дала 500 Ом, мінімум не змінився, а пару з -5 Ом програма пропустила.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.20.125.10.000 –Лр.4		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Чепель В.О.			Звіт з лабораторної роботи №4	Літ.	Арк. 1 / Акрушів
Перевір.		Лисенко М.С.					
Реценз.						ФІКТ, гр. КІ-26-1	
Н. Контр.							
Зав.каф.		Лобанчикова Н.М					

task1.c

```c
#include <stdio.h>

double paralel(double r1, double r2);
double min_opir(double r);

int main(void)
{
    int n;
    double r1, r2, r, rmin;

    printf("Скільки пар резисторів: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        printf("\nПара %d\n", i);
        printf("R1 = ");
        scanf("%lf", &r1);
        printf("R2 = ");
        scanf("%lf", &r2);

        if (r1 <= 0 || r2 <= 0) {
            printf("опір має бути більше 0\n");
            continue;
        }

        r = paralel(r1, r2);
        rmin = min_opir(r);

        printf("Опір: %.2lf Ом\n", r);
        printf("Мінімальний опір: %.2lf Ом\n", rmin);
    }

    return 0;
}

double paralel(double r1, double r2)
{
    return r1 * r2 / (r1 + r2);
}

double min_opir(double r)
{
    static double min = -1;

    if (min < 0 || r < min) {
        min = r;
    }
    return min;
}
```

Лістинг програми task1.c

```
PS D:\study\programming-sem1\lab04> gcc -Wall -Wextra task1.c -o task1
PS D:\study\programming-sem1\lab04> .\task1
Скільки пар резисторів: 4

Пара 1
R1 = 100
R2 = 100
Опір: 50.00 Ом
Мінімальний опір: 50.00 Ом

Пара 2
R1 = 47
R2 = 220
Опір: 38.73 Ом
Мінімальний опір: 38.73 Ом

Пара 3
R1 = 1000
R2 = 1000
Опір: 500.00 Ом
Мінімальний опір: 38.73 Ом

Пара 4
R1 = -5
R2 = 10
опір має бути більше 0
PS D:\study\programming-sem1\lab04>
```

Результат роботи програми

Завдання 2. У 32-бітному регістрі встановити біт 14, скинути біт 10, інвертувати біт 6 і перевірити стан біта 28.

Регістр зберігаю в unsigned int, на кожну операцію своя функція, маска робиться як 1u << n. Початкове значення 0xA5A5A5A5, бо там 14 біт нуль а 10 біт одиниця, і зміну видно одразу.

task2.c

```c
#include <stdio.h>

unsigned int set_bit(unsigned int reg, int n);
unsigned int clear_bit(unsigned int reg, int n);
unsigned int toggle_bit(unsigned int reg, int n);
int check_bit(unsigned int reg, int n);
void print_binary(unsigned int reg);

int main(void)
{
    unsigned int reg = 0xA5A5A5A5;

    printf("Початковий регістр:\n");
    print_binary(reg);

    reg = set_bit(reg, 14);
    printf("Встановлено біт 14:\n");
    print_binary(reg);

    reg = clear_bit(reg, 10);
    printf("Скинуто біт 10:\n");
    print_binary(reg);

    reg = toggle_bit(reg, 6);
    printf("Інвертовано біт 6:\n");
    print_binary(reg);

    printf("Біт 28 = %d\n", check_bit(reg, 28));
    printf("Кінцевий регістр: 0x%08X\n", reg);

    return 0;
}

unsigned int set_bit(unsigned int reg, int n)
{
    return reg | (1u << n);
}

unsigned int clear_bit(unsigned int reg, int n)
{
    return reg & ~(1u << n);
}

unsigned int toggle_bit(unsigned int reg, int n)
{
    return reg ^ (1u << n);
}

int check_bit(unsigned int reg, int n)
{
    if (reg & (1u << n)) {
        return 1;
    }
    return 0;
}

void print_binary(unsigned int reg)
{
    for (int i = 31; i >= 0; i--) {
        printf("%u", (reg >> i) & 1);
        if (i % 4 == 0) {
            printf(" ");
        }
    }
    printf("\n");
}
```

Лістинг програми task2.c

Якщо порівняти сусідні рядки, в кожному міняється тільки одна цифра, інші біти не чіпаються.

```
PS D:\study\programming-sem1\lab04> gcc -Wall -Wextra task2.c -o task2
PS D:\study\programming-sem1\lab04> .\task2
Початковий регістр:
1010 0101 1010 0101 1010 0101 1010 0101
Встановлено біт 14:
1010 0101 1010 0101 1110 0101 1010 0101
Скинуто біт 10:
1010 0101 1010 0101 1110 0001 1010 0101
Інвертовано біт 6:
1010 0101 1010 0101 1110 0001 1110 0101
Біт 28 = 0
Кінцевий регістр: 0xA5A5E1E5
PS D:\study\programming-sem1\lab04>
```

Результат роботи програми task2

Завдання 3. Упакувати в 16-бітне слово команду процесора: біти 0-3 код операції АЛП, біти 4-9 номер першого регістра, біти 10–15 номер другого регістра.

Код операції 4 біти, тому маска 0xF. Регістри по 6 біт, маска 0x3F, зсуви 4 і 10. В pack кожне поле обрізаю маскою, зсуваю вліво і додаю до слова через |. В unpack навпаки, зсуваю вправо і накладаю ту саму маску. Значень три, тому unpack повертає їх через покажчики. Код на рис. 5.

Для коду 11 і регістрів 37 і 5 вийшло 0x165B, це $5 \cdot 1024 + 37 \cdot 16 + 11 = 5723$. Після розпаковки вийшли ті самі числа. 15, 63, 63 дало 0xFFFF, а код 16 програма не прийняла

task3.c

```c
#include <stdio.h>

unsigned int pack(unsigned int op, unsigned int r1, unsigned int r2);
void unpack(unsigned int word, unsigned int *op, unsigned int *r1, unsigned int *r2);

int main(void)
{
    unsigned int op, r1, r2;
    unsigned int op2, r1_2, r2_2;
    unsigned int word;

    printf("Код операції (0..15): ");
    scanf("%u", &op);
    printf("Перший регістр (0..63): ");
    scanf("%u", &r1);
    printf("Другий регістр (0..63): ");
    scanf("%u", &r2);

    if (op > 15 || r1 > 63 || r2 > 63) {
        printf("значення поза діапазоном\n");
        return 1;
    }

    word = pack(op, r1, r2);
    printf("Слово: 0x%04X\n", word);

    unpack(word, &op2, &r1_2, &r2_2);
    printf("Після розпакування: op = %u, r1 = %u, r2 = %u\n", op2, r1_2, r2_2);

    if (op == op2 && r1 == r1_2 && r2 == r2_2) {
        printf("Значення збігаються\n");
    } else {
        printf("Значення не збігаються\n");
    }

    return 0;
}

unsigned int pack(unsigned int op, unsigned int r1, unsigned int r2)
{
    return (op & 0xF) | ((r1 & 0x3F) << 4) | ((r2 & 0x3F) << 10);
}

void unpack(unsigned int word, unsigned int *op, unsigned int *r1, unsigned int *r2)
{
    *op = word & 0xF;
    *r1 = (word >> 4) & 0x3F;
    *r2 = (word >> 10) & 0x3F;
}
```

Лістинг програми task3.c

```
PS D:\study\programming-sem1\lab04> gcc -Wall -Wextra task3.c -o task3
PS D:\study\programming-sem1\lab04> .\task3
Код операції (0..15): 11
Перший регістр (0..63): 37
Другий регістр (0..63): 5
Слово: 0x165B
Після розпакування: op = 11, r1 = 37, r2 = 5
Значення збігаються
PS D:\study\programming-sem1\lab04> .\task3
Код операції (0..15): 15
Перший регістр (0..63): 63
Другий регістр (0..63): 63
Слово: 0xFFFF
Після розпакування: op = 15, r1 = 63, r2 = 63
Значення збігаються
PS D:\study\programming-sem1\lab04> .\task3
Код операції (0..15): 16
Перший регістр (0..63): 1
Другий регістр (0..63): 1
значення поза діапазоном
PS D:\study\programming-sem1\lab04>
```

Результат роботи програми task3

Висновки: в цій лабораторній роботі було реалізовано три програми. В першій найменший опір вийшов 38,73 Ом (пара 47 і 220), його пам'ятає static змінна в функції min_opir. В другій після встановлення 14 біт, скинув 10 і інвертував 6, регістр став 0xA5A5E1E5, а 28 біт так і лишився нулем. В третій програмі числа 11, 37 і 5 запакувались в 0x165B, при розпаковці вийшли ті самі числа. Якщо в pack не обрізати поле маскою то регістр більший за 63 залізе в сусіднє поле, тому в main стоїть перевірка діапазону.